

Unit Testing

Unit Testing

- Getting Started
- Your First Unit Test
- Naming Conventions
- Running and Debugging Unit Tests
- Benefits of Unit Testing

What is Unit Testing?

- the process where you test the **smallest functional** unit of code.
- Hello.cs
 - Namespace
 - Class1
 - Method1=> unit Testing
 - Method2 => unit Testing
 - Method3 => unit Testing
 - Class2
 - class3
 - Namespace
 - Namespace

Ways to perform unit test

MSTest: If you're working on a **small or medium-sized project** and you want to stick to Microsoft's official tools, MSTest is a good option, especially if you're using Visual Studio.

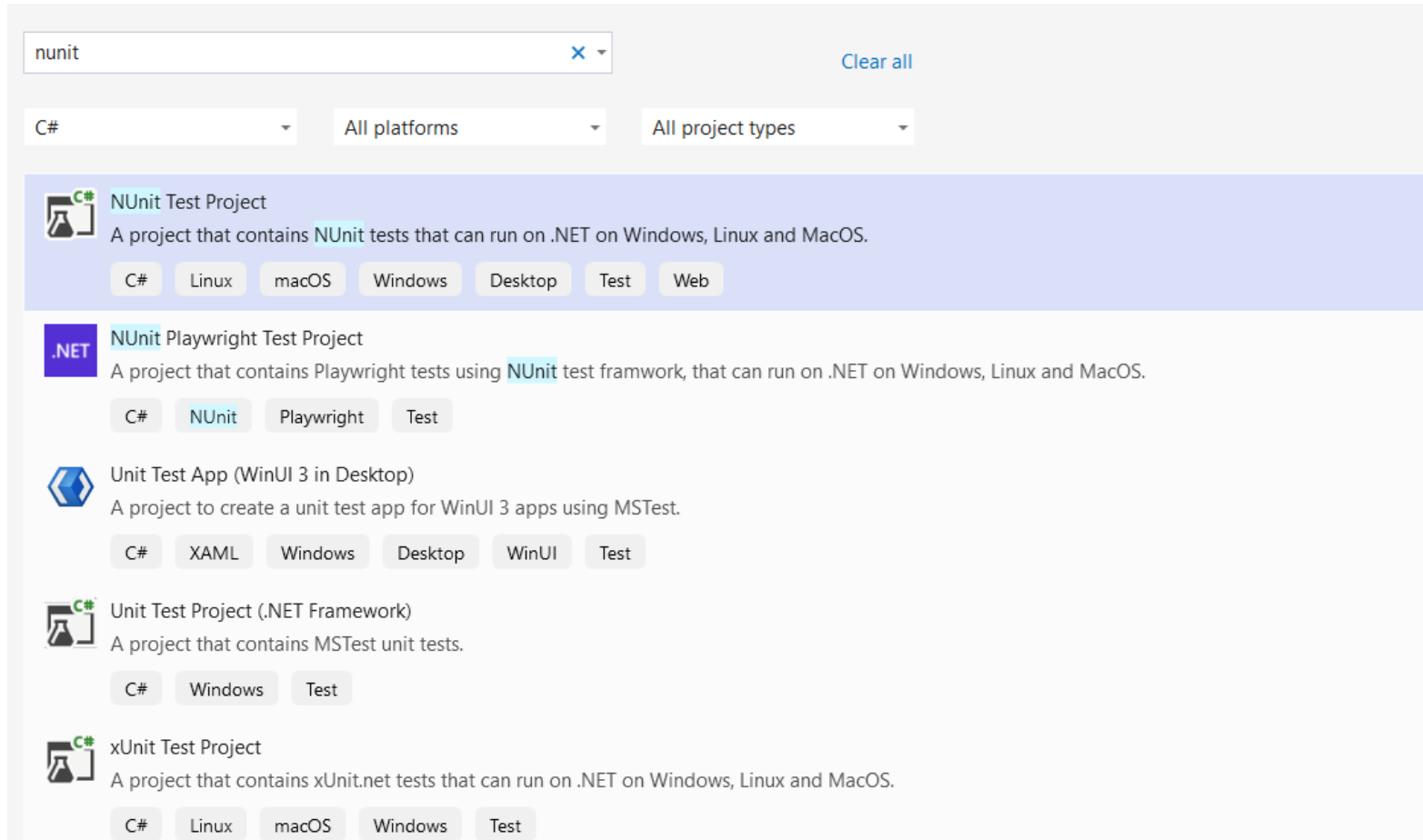
NUnit: Ideal if you need advanced features or **are working on legacy projects** that already use NUnit. It offers powerful assertions and flexibility in writing tests.

xUnit: The best choice for **modern .NET Core and .NET 5/6/7 projects**, or if you are following Test-Driven Development (TDD) or using dependency injection heavily in your tests. It encourages better testing practices and offers a cleaner, **more modern syntax**.

NUnit

- NUnit is one of the oldest and most widely used testing frameworks in .NET. It is independent of Visual Studio and has a rich set of features
- Is very developer-friendly with extensive attribute support.

Nunit Project Template



Nunit Project

```
public class Tests
{
    [SetUp]
    0 references
    public void Setup()
    {
    }

    [Test]
    0 references
    public void Test1()
    {
    }
}
```

Assert

- *Assert* is a static class which contains some methods to help you make checks in your test code.
- statements or conditions that verify specific expectations about the behavior of the code being tested

Tests in group: 1
Total Duration: 2 ms

13  

`Assert.AreEqual()`

▲ 1 of 6 ▼ `void Assert.AreEqual(object? expected, object? actual)`

Verifies that two objects are equal. Two objects are considered equal if both are null, or if both have the
expected: *The value that is expected*

TestCase

[Test]

[TestCase(10,10,20)]

```
public void Test1(int x, int y, int z)
```

```
{
```

```
    int r = x + y;
```

```
    Assert.AreEqual(z, r);
```

```
}
```

```
[Test]
[TestCase(10,10,new int[] { 20,2,3 })]
public void Test1(int x, int y, int[] z)
{
    int r = x + y;

    Assert.AreEqual(z[0], r);
}
```

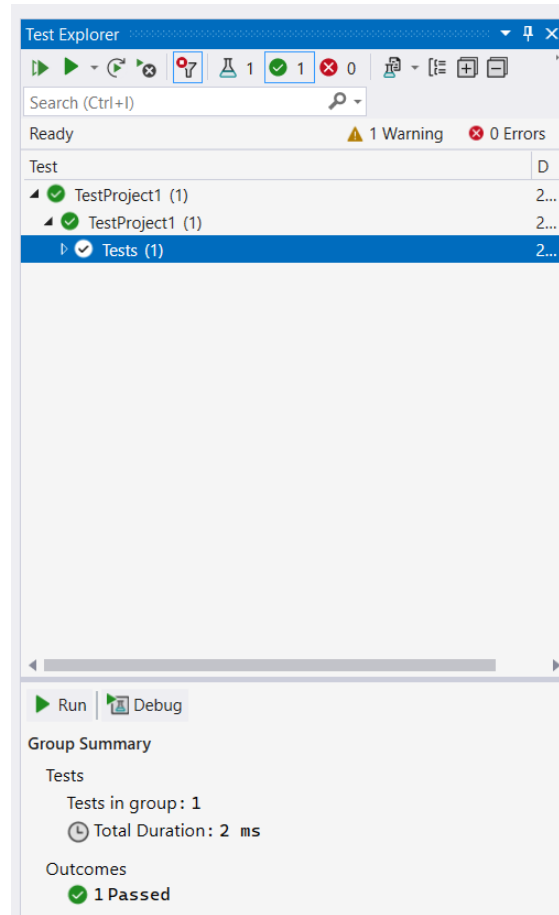
Attributes

Attributes	Descriptions	Scenarios
Setup	Runs before each test to initialize any resources needed by that test.	Check Authentication
[Test]	Marks a test method.	Area to run the test
[Teardown]	Runs after each test to clean up or release resources used by that test.	Logging
[OneTimeSetUp]	Runs once before all the tests in a test suite/class to initialize resources needed by all tests	Database connection/FilePath
[OneTimeTeardown]	Runs once after all the tests in a test suite/class to clean up shared resources initialized during OneTimeSetup.	Close Database connection/ any resource needs to cleanup
[Ignore]	Skips Test Method	If you do not want to test method

Steps Involved in Unit Test

- Step-1
 - Develop a code using Library or any Application
- Step-2
 - Create Unit Test Project
- Step-3
 - Inside Test Method Unit Test is performed
- Step-4
 - You can View result in TestExplorer Window

Test Explorer Window



Common Assert Methods

- `Assert.AreEqual(object, object)`
- `Assert.IsTrue(bool)`, `Assert.IsFalse(bool)`
- `Assert.IsNull(object)`, `Assert.IsNotNull(object)`
- `Assert.ThrowsException(Action)` — useful if you expect an exception to be thrown

